Recursivitat (amb torres de Hanoi)



Algorísmica

Grau d'Enginyeria Informàtica

Mireia Ribera, ribera@ub.edu



Recursivitat: Conceptes clau

1. Motivació
2. Definició i anatomia: cas base i reducció
3. Cost i eficiència: pila d'execució. Versió iterativa



Motivació

Ja hem vist que una funció en pot cridar a una altra... però té sentit que *una funció es cridi a si mateixa?*

Veiem un exemple: el càlcul del factorial.

$$n! = \begin{cases} 1 & \text{si } n = 0, \\ 1 & \text{si } n = 1, \\ n \cdot (n-1)! & \text{si } n > 1 \end{cases}$$



Definició

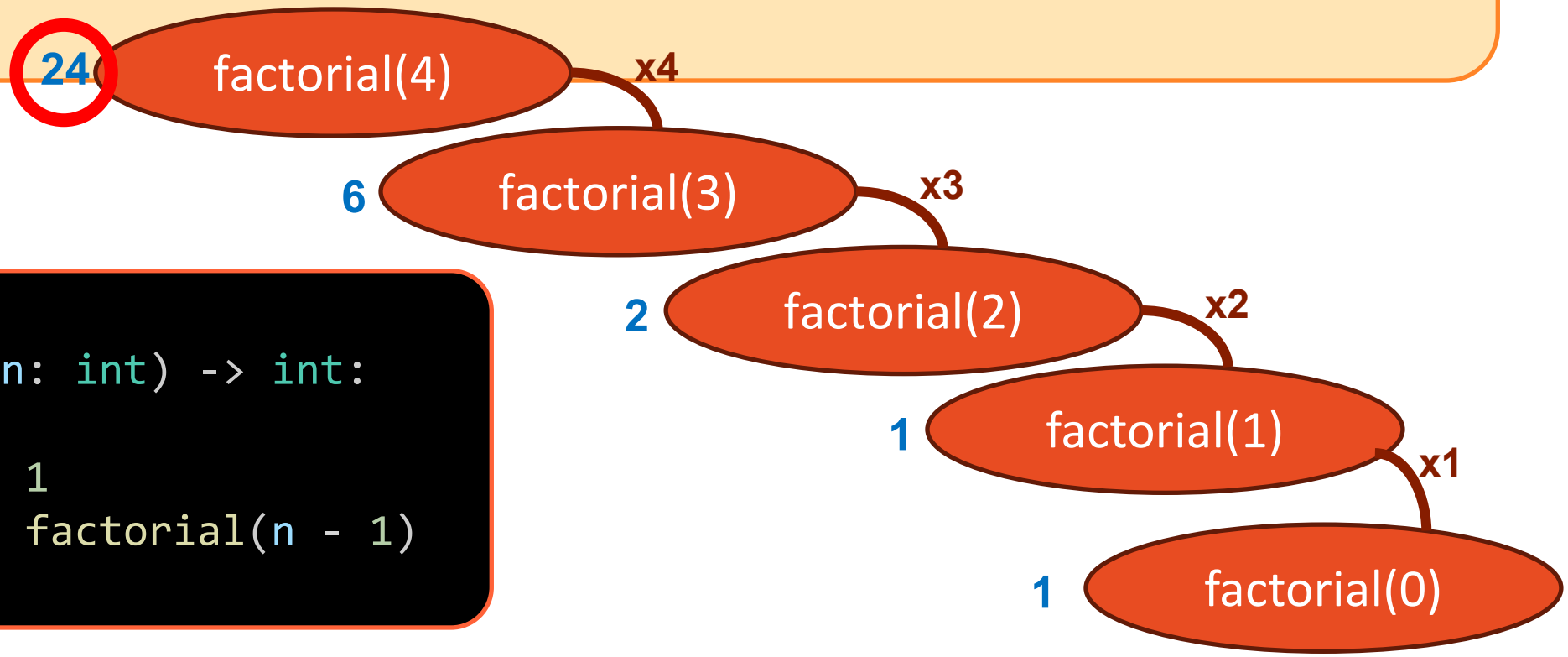
La recursivitat és una tècnica de resoldre un problema computacional en el que la solució depèn de solucions a instàncies més petites del mateix problema. Font: Wikipedia

El problema es redueix a versions més petites fins a arribar a un cas molt senzill que podem resoldre directament.

Exemple de càlcul del factorial

La recursivitat és una tècnica de resoldre un problema computacional en el que **la solució depèn de solucions a instàncies més petites** del mateix problema.

El problema es redueix a versions més petites **fins a arribar a un cas molt senzill** que podem resoldre directament.




```
def factorial(n: int) -> int:
    if n == 0:
        return 1
    return n * factorial(n - 1)
```



Anatomia d’una funció recursiva

🧠 Quan resolem un problema amb recursivitat ens cal definir:

1. 🛑 El cas base: la condició que atura la recursió.
2. 🔁 La crida recursiva: com avancem cap al cas base, fent el problema més petit cada vegada.

⚠️ Errors típics:

- Oblidar el cas base: l’execució no pararà mai, serà infinita.
- No reduir el problema correctament.
- Pensar només en el primer pas sense tenir clar tot el problema.

Ara et toca... (Exercici1)



Escriu una funció que calculi x^n amb recursivitat. x i n enters.
Defineix quin és el cas base i com vas reduint el problema a cada crida.

La recursivitat no sempre és la millor solució... (1)

Observem què passa amb les crides de Fibonacci

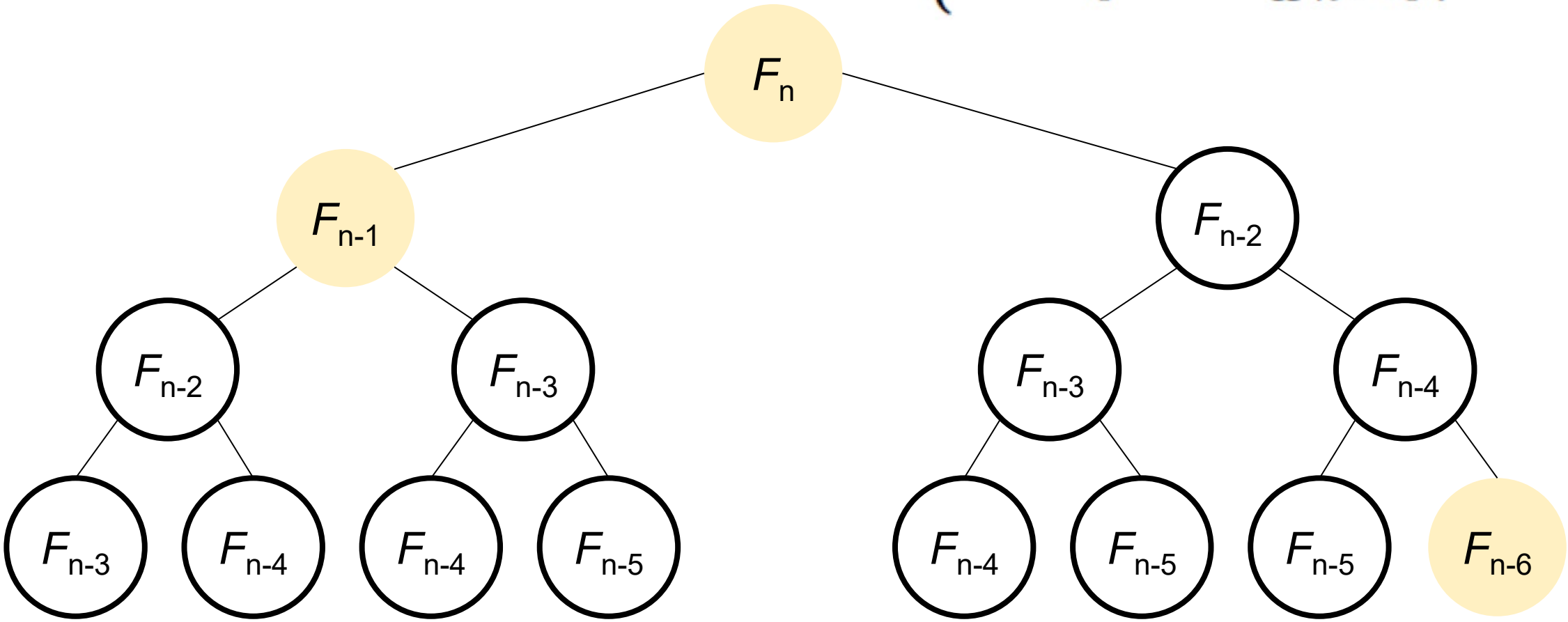
$$F_n = \begin{cases} F_{n-1} + F_{n-2} & \text{if } n > 1 \\ 1 & \text{if } n = 1 \\ 0 & \text{if } n = 0\,. \end{cases}$$

```python
def fibonacci(n: int) -> int:
    if n == 0:
        return n
    if n == 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
```

La recursivitat no sempre és la millor solució... (2)

Observem què passa amb les crides de Fibonacci

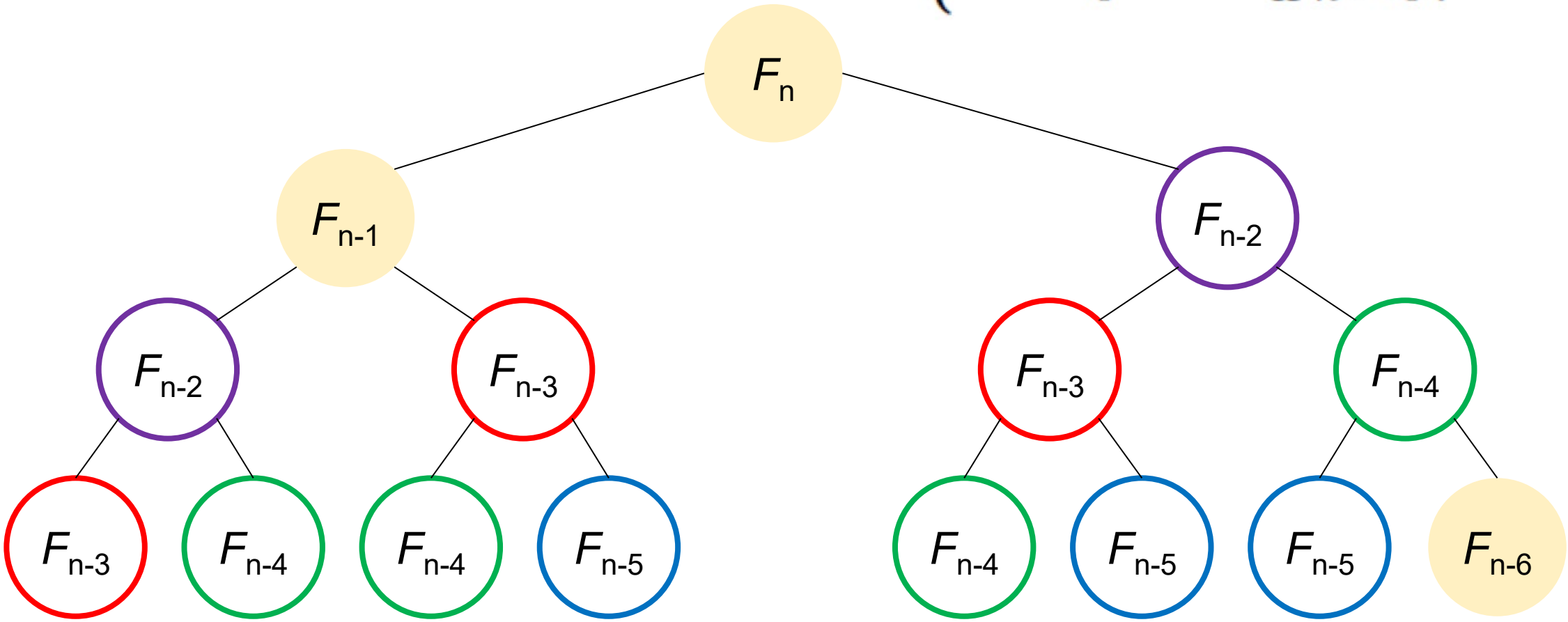
$$F_n = \begin{cases} F_{n-1} + F_{n-2} & \text{if } n > 1 \\ 1 & \text{if } n = 1 \\ 0 & \text{if } n = 0\,. \end{cases}$$



La recursivitat no sempre és la millor solució... (3)

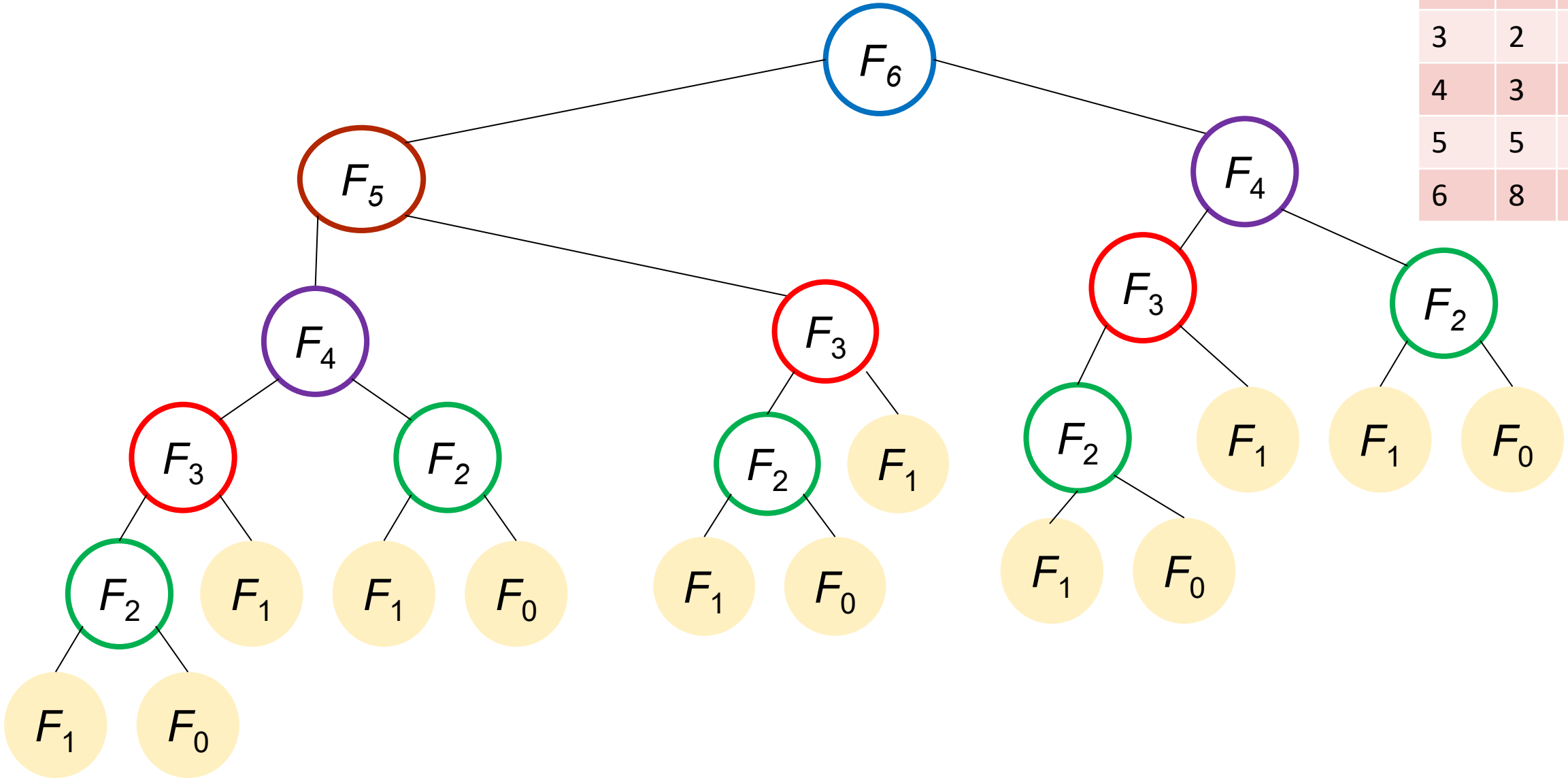
Observem què passa amb les crides de Fibonacci

$$F_n = \begin{cases} F_{n-1} + F_{n-2} & \text{if } n > 1 \\ 1 & \text{if } n = 1 \\ 0 & \text{if } n = 0\,. \end{cases}$$



Quants casos base hi ha per cada pas?

n	F	c.b
0	0	0
1	1	0
2	1	2
3	2	3
4	3	5
5	5	8
6	8	13





Podem millorar l'eficiència...

```python
def fibonacci_iteratiu(n: int) -> int:
    if n == 0:
        return 0

    seq: List[int] = [0, 1]
    for i in range(2, n + 1):
        seq.append(seq[i - 1] + seq[i - 2])

    return seq[n]
```

```python
def fibonacci_iteratiu_optimitzat(n: int) -> int:
    a: int = 0
    b: int = 1

    for i in range(1, n + 1):
        a, b = b, a + b

    return a
```



Fins i tot quan no hi ha repeticions...

```python
def factorial(n: int) -> int:
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

- La recursivitat ajuda a conceptualitzar un problema de manera elegant.
- Cal assegurar-se de definir el cas base i que a cada pas ens hi apropem.
- L'execució no sempre és eficient, si hi ha opció iterativa cal valorar avantatges i inconvenients tenint molt en compte l'eficiència.



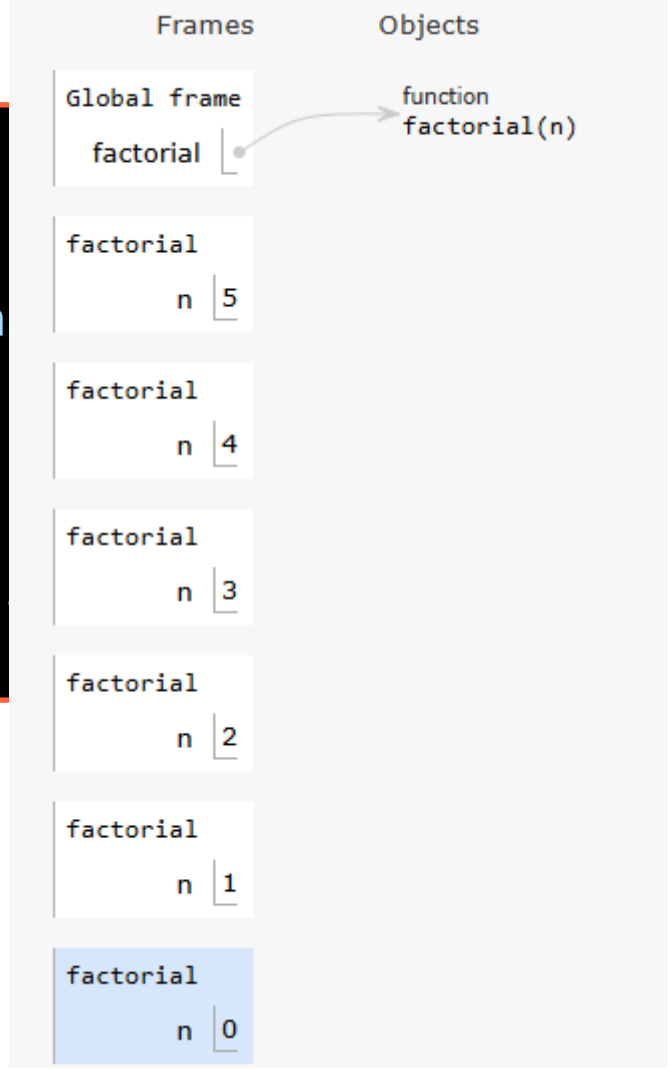
Què esta passant dins l'ordinador?

```python
# recursiu
def factorial(n: int) -> int:
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

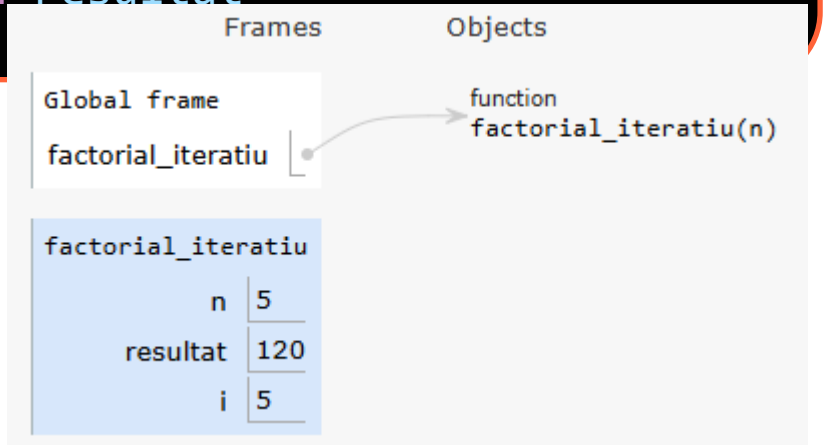
```python
# iteratiu
def factorial_iteratiu(n: int) -> int:
    resultat = 1
    for i in range(1, n + 1):
        resultat *= i
    return resultat
```

Què esta passant dins l'ordinador? (1)

```
# recursiu
def factorial(n
    if n == 0:
        return
    return n *
```




```
# iteratiu
def factorial_iteratiu(n: int) -> int:
    resultat = 1
    for i in range(1, n + 1):
        resultat *= i
    return resultat
```



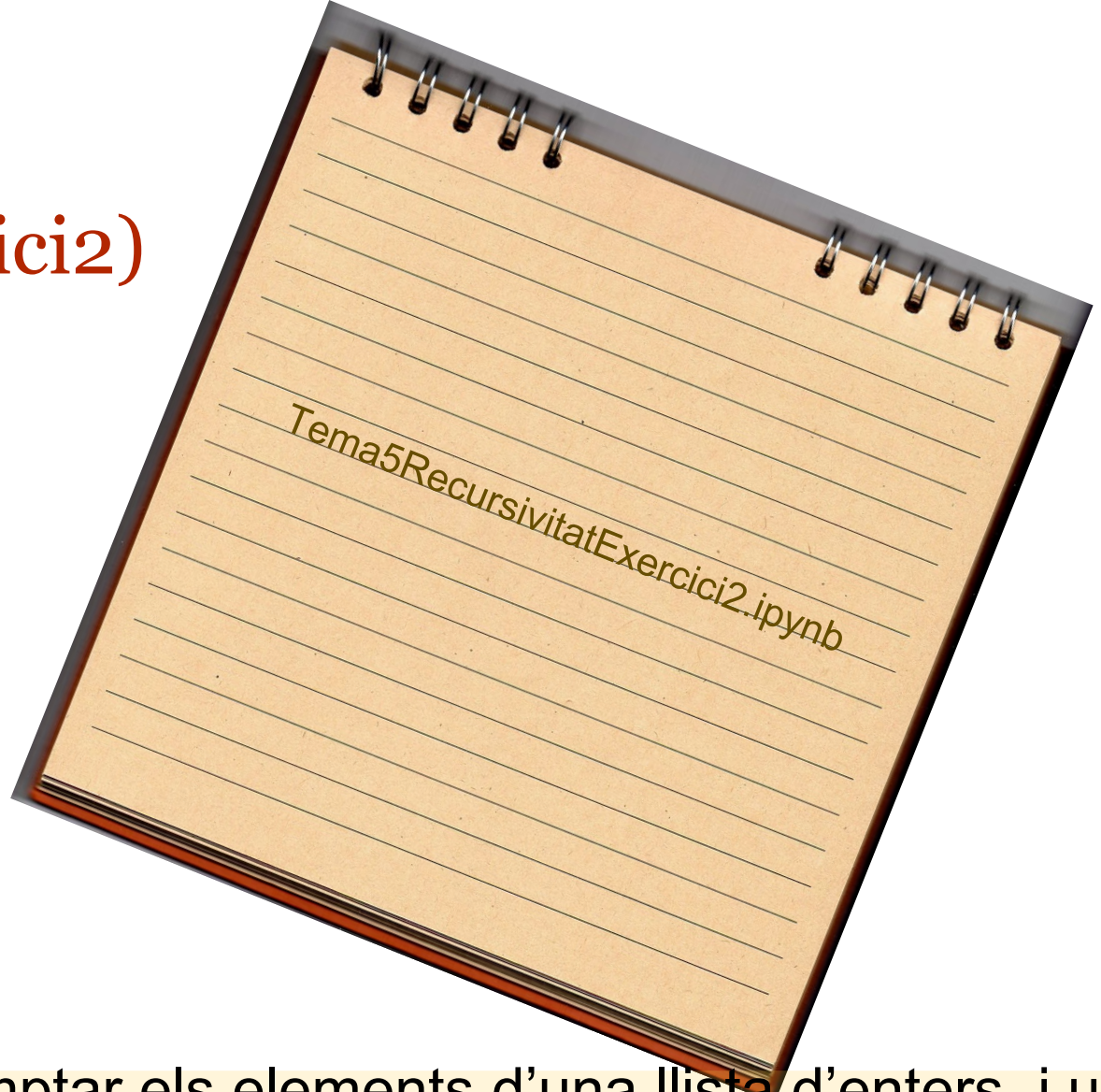


Notebook de suport: Fibonacci



Compara les tres versions de Fibonacci: RecursiusTeoriaFibonacci.ipynb

Ara et toca... (Exercici2)



Escriu una funció per comptar els elements d'una llista d'enters, i una altra per revertir una cadena. No pots fer servir funcions de Python.
De cadascuna fes-ne una versió recursiva i una iterativa i calcula'n el cost.
Pots inspirar-te en el següent notebook de suport, suma d'una llista.

Més difícil...



Escriu una funció que donat un nombre enter retorni una cadena amb nombre parèntesis ben formats. Per exemple parèntesi(3) retornarà “((()))” fes-ne la versió recursiva i la iterativa.



Notebook de suport: TeoriaSumaLlista



Un exemple complet, la suma d'una llista: RecursiusTeoriaSumaLlista.ipynb

Gràcies i recapitulació (1)





UNIVERSITAT DE BARCELONA



Recapitulació (1)

✅ **Avantatges de la recursivitat**

✨ Codi més curt, més expressiu

📐 Més proper a la definició matemàtica

⚠️ **Inconvenients de la recursivitat**

💾 Sobrecàrrega de memòria (per la pila d'execució)

⏳ Sovint ofereix menor rendiment

Les versions iteratives dels algorismes solen ser més eficients, tot i que a vegades són menys elegants. Cal valorar avantatges i inconvenients en cada cas.



Torres de Hanoi: Un problema recursiu intractable

1. Origen
2. Definició del problema
3. Resolució

Origen

El joc matemàtic de les Torres de Hanoi va ser inventat pel matemàtic francès Édouard Lucas l'any 1883.

"Segons una llegenda índia, al Temple de Benarés, sota el punt que marca el centre del món, hi ha una placa de llautó amb tres agulles de diamant.

Durant la creació, Déu va situar 64 discs d'or pur de diferents mides en una de les agulles, formant una torre.

Els monjos porten generacions canviant de lloc, un a un, els discs de la torre entre les tres agulles de forma que mai un disc més gran es trobi sobre un més petit.

Quan acabin de traslladar tots els discs a una altra agulla hauran acabat la feina, i la torre i el temple s'esfondraran, provocant un gran tro que farà desaparèixer el món."

Font del text: MMACA https://mmaca.cat/moduls/torres-hanoi/



Temple Tulsi Manas Mandir, Varanasi

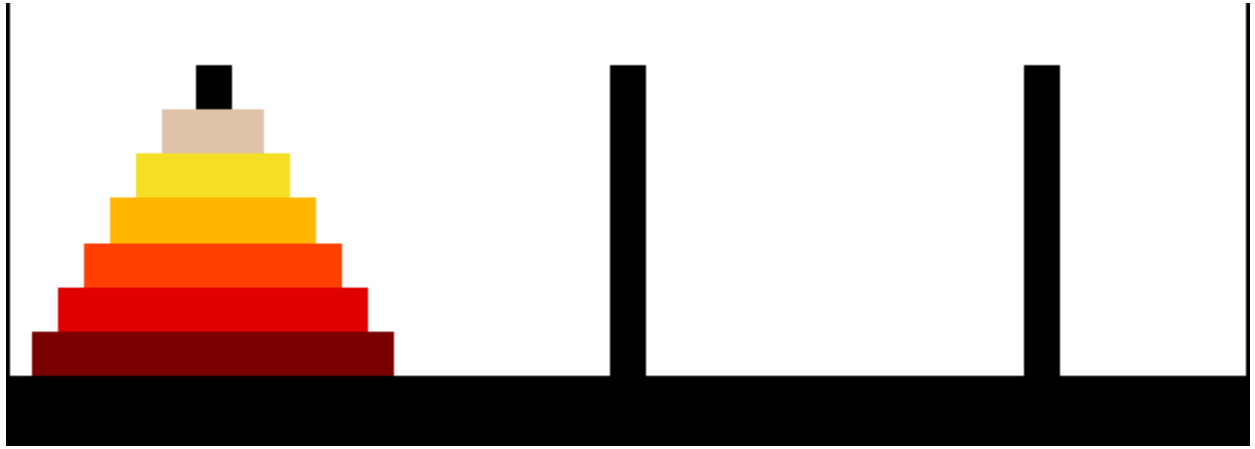


Font de la imatge d'Édouard Lucas: Wikipedia

Torres de Hanoi: definició del problema

Tenim tres varetes i diversos discs. A l'inici els discs estan col·locats de més gran a més petit en la primera vareta.

El joc consisteix en passar tots els discs a la tercera vareta tenint en compte que només es pot canviar de vareta un disc cada vegada i que mai no podem tenir un disc col·locat sobre un que sigui més petit.



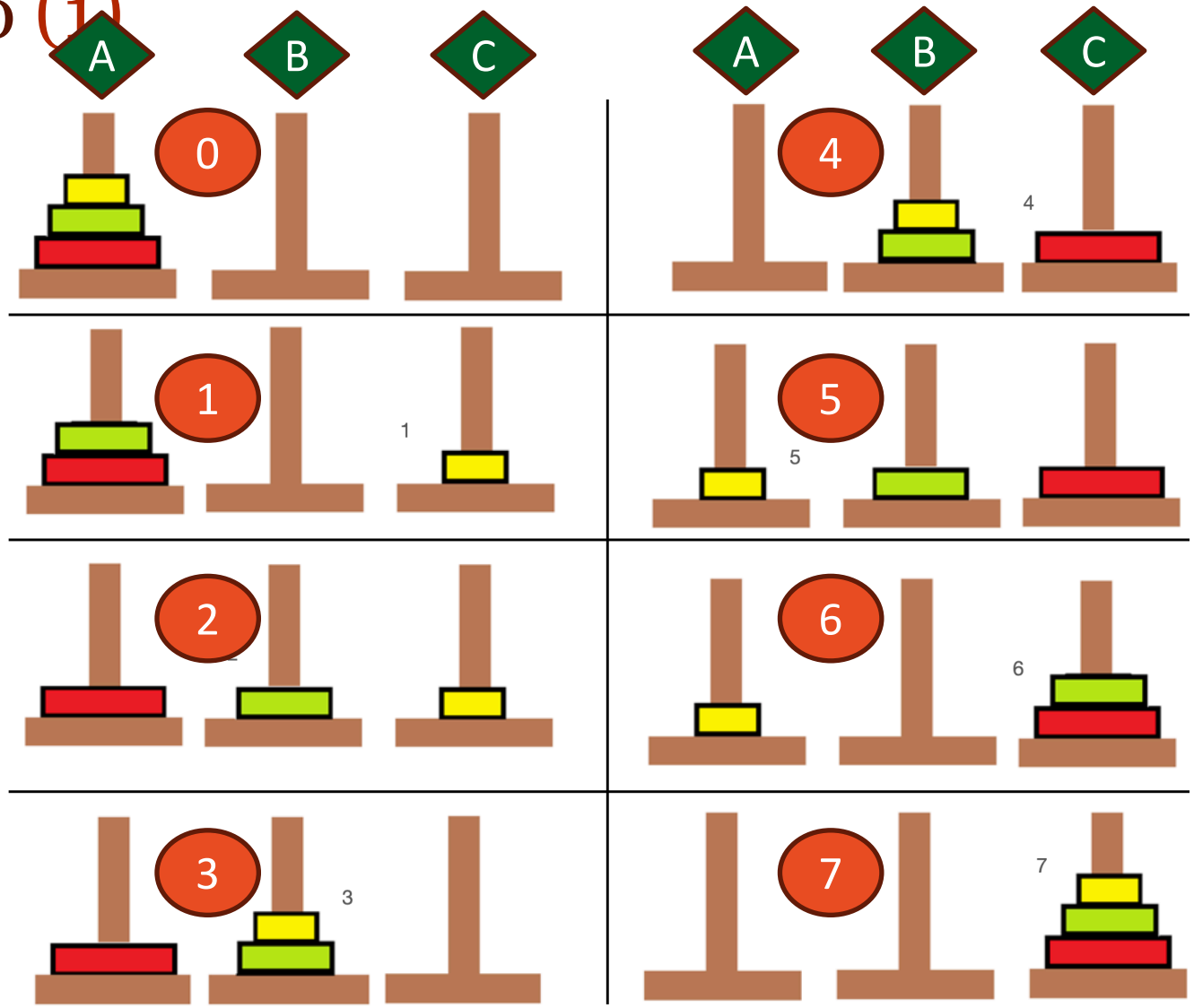
Ara et toca... (joc)



Juga al joc online i prova d'arribar a resoldre una torre amb 5 discs.
Posa en funcionament el pensament algorítmic per trobar un patró.

Torres de Hanoi: resolució (1)

- Pas 0 a 4 moure disc gran a C, amb l'ajut de B.
  - 1-3 Moure discs que estan a sobre de A a B amb l'ajut de C
- Pas 5 i 6 moure següent disc gran no col·locat de B a C amb l'ajut d'A.
- Pas 7 moure últim disc a C.





Resolució codi...

Necessitem una funció bàsica per moure un disc d'una vareta a una altra.

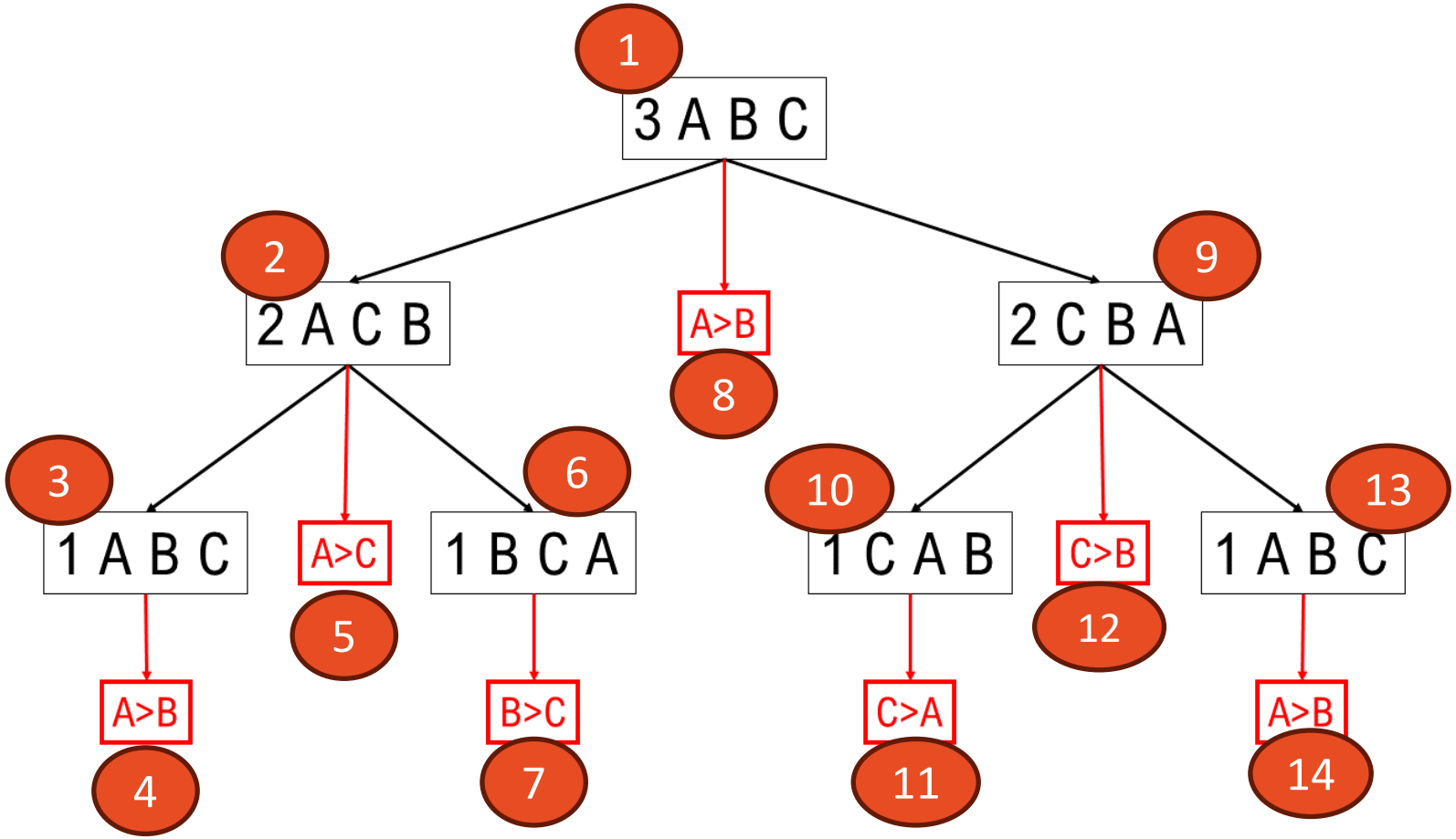
I una funció que sàpiga què fer quan hi ha més d'un disc...

```
def moure_disc(dv:str,tv:str)->None:
    print("movent disc de", dv, "to", tv)
moure_torre(3, "A", "B", "C")
```

```
def moure_torre(alcada:int, de_vareta:str, a_vareta:str,
amb_vareta:str)->None:
    if alcada == 1:
        moure_disc(de_vareta, a_vareta)
    else:
        ...
```

Torres de Hanoi: resolució (2)

El problema es resol amb crides recursives, i el cas base és quan només cal moure un disc.



* 3ABC vol dir que hi ha tres discs a A, que es mouran a B amb l'ajut de C.

Resolució codi (2)...

Vegem el codi sencer de la segona funció

```
def moure_torre(alcada:int, de_vareta:str, a_vareta:str, amb_vareta:str)->None:
    if alcada == 1:
        moure_disc(de_vareta, a_vareta)
    else:
        moure_torre(alcada-1, de_vareta, amb_vareta, a_vareta)
        moure_disc(de_vareta, a_vareta)
        moure_torre(alcada-1, amb_vareta, a_vareta, de_vareta)
```



Torres de Hanoi. Complexitat

Què passaria amb l'arbre si enlloc de 3 peces en tenim 4?

Quantes vegades es crida recursivament la funció (quantes fulles i nodes te l'arbre)

si tenim n peces?

Quina és la complexitat de l'algorisme?

Gràcies i recapitulació (2)

MIREIA RIBERA | ACCESSIBILITAT DIGITAL
| EXPERIÈNCIA D'USUARI
|VISUALITZACIÓ DE DADES

UNIVERSITAT DE BARCELONA



Recapitulació (2)

⏳ Les torres de Hanoi és un problema recursiu NP (temps polinomial no determinista). Pots verificar la solució però no és fàcil trobar-la.

✨ La resolució recursiva és clara i sintètica.

⚠️ Amb pocs discs el problema creix molt ràpidament.

I de fet, a la llegenda, si tarden un segon a moure cada disc, amb 64 discs els monjos tardaran

585 bilions d'anys

a moure tots els discs. 😅 No cal que patim encara per la fi del món!